

SOEN 203 – Evolution of Database Systems & ER Modeling

Kisii University | Clean Revision Notes

1. Evolution of Database Systems

Database systems evolved from simple file storage to highly scalable, distributed architectures. The main stages recorded in the notes are summarised below.



Figure: Evolution of Database Systems (from notes)

1.1 Ancient Times & File-based Systems

In the earliest period **data was stored in ordinary files** (file-based system / file-based processing). Each application managed its own files independently.

Disadvantages of the File-based Approach

- Uncompatible / isolated data – different applications' files could not easily work together.
- Data redundancy – the same facts were often stored in many places.
- Not easily accessible – retrieval required custom programs for each file.
- Not suitable for large institutions – hard to keep consistency and concurrent access.
- Performance and maintenance problems grew with volume.

1.2 Database Approach

Data is stored in a shared, organised repository managed by a Database Management System (DBMS). This reduces redundancy, improves consistency and makes the same data available to many applications.

1.3 Hierarchical Model (① Early 1960s)

The first structured database model. Data is organised as a **tree**: each child record has exactly one parent. Simple and fast for hierarchical data, but inflexible when relationships are not strictly one-to-many.

1.4 Network Model (② Mid-1960s)

Allowed more flexibility because a record can have **multiple parent records**, creating a network-like (graph) structure. More powerful than hierarchical, but also more complex to design and navigate.

1.5 Relational Model / SQL (③ 1970s – present)

Offered a **simple and more flexible** way to organise and retrieve data using **tables, rows and columns**. Data is manipulated with Structured Query Language (SQL). It remains the dominant model for most business applications.

Examples (from notes): MySQL, Oracle, Microsoft SQL Server, Access, DB2, PostgreSQL, SQLite.

1.6 Object-Oriented Models (1980s–1990s)

Designed to store **complex data types and relationships**. Meant to bridge the gap between the relational model and object-oriented programming by incorporating **encapsulation** and **inheritance**.

Limitation noted: often complex, with lower performance for typical business queries.

1.7 Object-Relational → NoSQL

Object-relational systems tried to combine OO features with relational foundations. Later, **NoSQL databases** appeared, designed for high scalability and flexibility with large volumes of unstructured data.

Type	Example(s)
Key-Value store	Redis, Dynamo
Document store	MongoDB
Column-family store	Cassandra
Graph database	ArangoDB

1.8 NewSQL, Cloud & Distributed Databases (2010s)

NewSQL combines NoSQL-style scalability with the ACID guarantees and SQL interface of traditional relational systems. A noted drawback is that some systems may trade consistency for availability under certain conditions.

Cloud & Distributed databases (including blockchain-related designs) are built for cloud environments with high availability and a distributed architecture, supporting horizontal scaling across many machines.

Modern social applications (WhatsApp, Instagram, Facebook) rely on highly scalable backends – often a mix of relational stores for structured data and NoSQL/document stores for feeds, messages and unstructured content.

2. Attributes – Types & Properties

Single-valued vs Multi-valued

- **Single-valued** – holds only one value (e.g. Gender).
- **Multi-valued** – can hold more than one value (e.g. PhoneNumber: 01-..., 07-...).

Derived attributes

A value calculated from other attributes and **not stored permanently**. Example: Age is derived from Date of Birth and is not assigned a stored value of its own.

Required vs Optional

- **Required** – must be present; the constraint is enforced (NOT NULL).
- **Optional** – the user may leave the field empty.

Domain

The set of all possible legal values for a given attribute. Example: domain of Gender = {Male, Female, Other}.

3. Keys

Keys are attributes (or sets of attributes) used to identify and relate entities.

Superkey — A set of attributes that uniquely identifies each entity. It includes the primary key (and possibly more). Examples: RegNo, ID number, Phone number, Email address.

Candidate Key — A minimal set of attributes from which a primary key can be chosen. Example: RegNo and ID number are both candidate keys for a Student.

Primary Key — The candidate key chosen to uniquely identify each entity (e.g. RegNo).

Alternate Key — Any candidate key that was not chosen as the primary key.

Foreign Key — A key placed in one table to reference the primary key of another table. It connects (relates) two entities.

4. Entity Representation & Relations

A common graphical style is the **Crow's-foot** (dual-section box) model: a header for the entity name and a body listing attributes. The primary-key attribute is usually marked (PK).



Figure: Simple entity box (entity name + attributes)

Relation — a set of meaningful associations between particular entities. Example from notes: Student *owns* Course; Department *offers* unit; Employee always belongs to a department.

5. DBMS Architecture

Architecture is crucial for efficient data management and system performance. The notes distinguish three classic tiers.

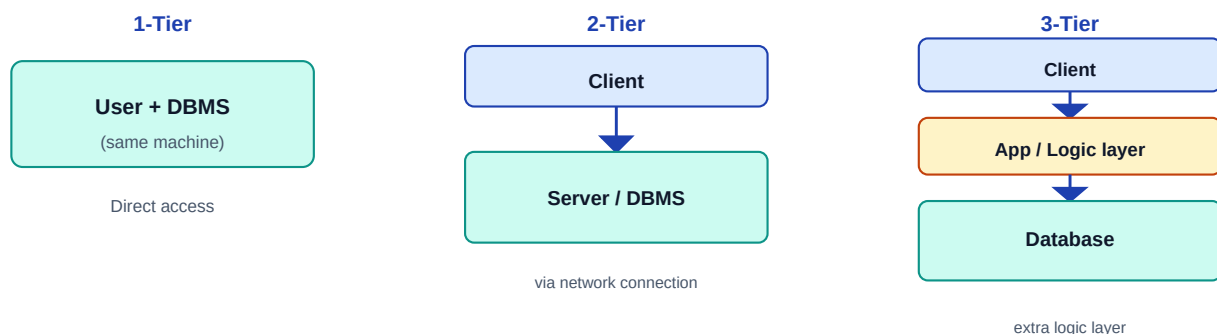


Figure: DBMS Architecture – 1-tier, 2-tier, 3-tier

1-Tier Architecture

The user works directly on the same machine that runs the DBMS. There is no network separation. Typical of single-user desktop databases (e.g. a local Access or SQLite file opened by the user).

2-Tier Architecture

The client (application or query tool) interacts with the DBMS server over a network connection. The client sends SQL; the server executes it and returns results. Classic client-server model used by many departmental systems.

3-Tier Architecture

An extra middle layer (application / business-logic server) sits between the client and the database. The client talks only to the application server; the application server talks to the database. This separation improves security, scalability and maintainability. Typical of modern web applications: Browser → Web/Application Server → Database Server.

6. Entity-Relationship (ER) Modeling

ER modeling is a **top-down** approach to database design. The first step is to identify the **entities** and the relationships that must be represented. Further information about each entity (its characteristics/attributes) is then identified, followed by the relations between attributes.

Entity

A set of objects that share the same characteristics. An **entity occurrence** (instance) is one concrete object identifiable within that set. Nouns in a problem description usually point to entities. Entity names should be unique within the model.

Strong & Weak Entities

A **strong entity** can be uniquely identified by its own attributes. A **weak entity** cannot; it depends on a strong (owner) entity for identification. In the notes: Parent = strong, Child = weak; Employee = strong, Dependent = weak.



Figure: Strong entity (Employee) and dependent weak entity (Dependent)

Why an ATM card is a weak entity (explanation requested in the notes): An ATM/debit card cannot be fully identified or understood on its own. Its existence and identity depend on the owning Account (or Customer). Without the account context the card number has no independent meaning in the bank's data model; therefore the card is modelled as a weak entity that depends on the strong Account entity.

Simple & Composite Attributes

- **Simple** – contains a single component that cannot be subdivided further (e.g. Price).
- **Composite** – can be subdivided into smaller meaningful parts (e.g. Full Name → First Name + Last Name).

7. Degree of a Relationship

The degree of a relationship is the number of entity types that participate in it.

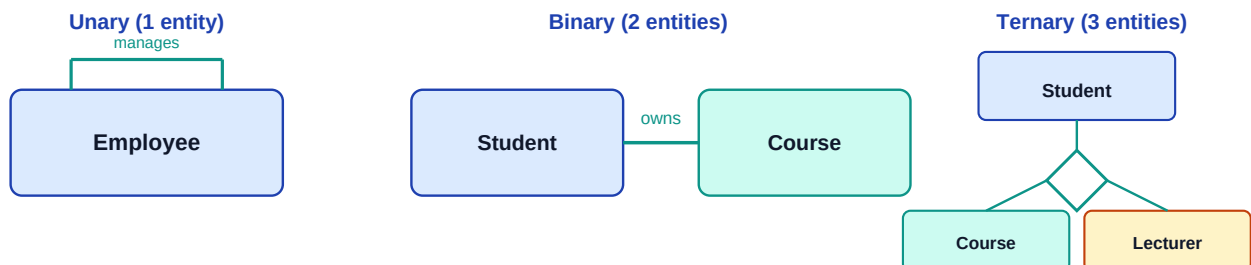


Figure: Degree of relationships – Unary, Binary, Ternary

- **Unary** – one entity participates with itself (self-relationship). Example: Employee manages Employee.
- **Binary** – relationship between two different entity types. Example: Student owns Course.
- **Ternary** – relationship involving three entity types simultaneously. Example: Student – Course – Lecturer.

Quick Revision Checklist

- File-based → Hierarchical (tree) → Network (many parents) → Relational (tables) → OO → NoSQL → NewSQL / Cloud.
- Relational model uses tables, rows, columns and SQL; still dominant for structured business data.
- NoSQL types: Key-Value, Document, Column-family, Graph – built for scale and unstructured data.
- Keys: Superkey ⊃ Candidate Key ⊃ Primary Key; Alternate = unused candidate; Foreign Key links tables.
- Derived attribute (e.g. Age) is calculated, not stored; Domain = allowed values for an attribute.
- Strong entity stands alone; Weak entity needs an owner (e.g. Dependent needs Employee; ATM card needs Account).
- 1-tier = user + DBMS together; 2-tier = client ↔ server; 3-tier = client ↔ app logic ↔ database.
- Degree: Unary (self), Binary (2 entities), Ternary (3 entities).
- ER modelling is top-down: identify entities → relationships → attributes.